B.Tech Project Report

Urban Microclimate Modelling via Neural Surrogates and Computational Fluid Dynamics (CFD)

A comparative study of a morphometric Random Forest baseline and convolutional neural surrogates for pedestrian-level urban wind prediction

Submitted by

Team T095 | Section B

B.Tech, Computer Science and Artificial Intelligence | 3rd Year, 5th Semester

Report prepared: 28 October 2026

Abstract

Fast prediction of pedestrian-level wind in cities is useful for urban design, but high-fidelity CFD and large-eddy simulation (LES) are too slow for interactive exploration. This project builds a complete, testable software platform for comparing two surrogate strategies: a traditional machine-learning baseline that predicts a single area-averaged wind speed from 26 morphometric features using a Random Forest, and spatial neural surrogates (a residual Compact U-Net and a dilated Compact CNN) that predict a full 128 x 128 wind-speed field from a 7-channel raster of geometry and inflow conditions.

The implementation includes a data contract, rasterizer and feature extractor, layout-level and family-level splitters, train-fitted normalisers, outdoor-masked losses and metrics, a PyTorch trainer, a FastAPI service with inventory endpoints, and a canvas-based research frontend. The project documents report 207 passing automated tests, but the test breakdown remains unresolved (Section 6.1). Some frontend integrations and post-change end-to-end behaviour have not been independently verified, as documented in Table 6.4.

The project's most important outcome is methodological. An early backend audit found that the API labelled analytical fields as OpenFOAM output and produced neural "predictions" by perturbing the target field instead of running the networks. Both defects were removed (P0), a provenance-aware inventory layer was added (P1), and every claim in this report is tied to what the repository can actually support.

What the results do and do not show. No real external dataset (UrbanTALES) was obtained and no OpenFOAM case was executed, so no result here is a measurement against real LES or CFD data. The reported checkpoints are untrained, so the comparison on the showcase layout (Random Forest MAE 1.127 m/s; untrained U-Net 2.608 m/s; untrained CNN 2.909 m/s) validates the evaluation pipeline only. It is not a fair comparison of trained models, and the Random Forest's held-out status on that layout is unverified. The report therefore presents a verified, honest evaluation framework and a clear route to a real scientific result once data and solver access are available.

Keywords: urban microclimate, pedestrian wind, neural surrogate, U-Net, Random Forest, CFD, UrbanTALES, scientific integrity.

1. Introduction

1.1 Background and motivation

Wind at pedestrian height shapes comfort, safety and the dispersion of pollutants in streets. It is governed by building layout: dense blocks, street canyons and tall towers create stagnation zones, accelerated corner jets and recirculating wakes. Resolving these effects normally needs CFD (Reynolds-averaged simulation, RANS) or LES. Each simulation can take hours on a cluster, which makes it impractical to explore hundreds of design variants during early planning.

A neural surrogate learns the mapping from a layout and inflow condition to a flow field from a library of simulations. Once trained, inference takes milliseconds. The scientific questions are how accurate such a surrogate is, how it generalises to unseen layouts, and whether spatial networks justify their cost over a cheap tabular baseline.

1.2 Problem statement

Build and rigorously evaluate a platform that predicts pedestrian-level (z = 1.75 m) wind speed over a 2D urban layout, comparing (i) an applied-ML baseline that predicts one scalar per layout with (ii) spatial neural surrogates that predict the whole field, under a leakage-free evaluation protocol and with transparent data provenance.

1.3 Objectives

1. Define a strict data contract and ingestion path that a real dataset such as UrbanTALES can plug into without competing abstractions.
2. Implement a 26-feature morphometric extractor and a 7-channel rasterizer feeding a Random Forest baseline and two convolutional surrogates.
3. Evaluate only over outdoor fluid cells, using layout-level or family-level holdout, with train-only normalisation.
4. Expose the system through a REST API and a research frontend that never mislabels synthetic data as solver output.
5. Audit the project honestly: identify and remove unsupported claims, and document what remains blocked.

1.4 Scope and non-goals

- **In scope:** 2D horizontal slices at z = 1.75 m, neutral isothermal boundary layer, time-averaged wind speed.
- **Out of scope:** air temperature, solar radiation, thermal buoyancy and any 3D volumetric prediction. The dataset audit confirms UrbanTALES is isothermal, so temperature cannot be predicted from it. If an examiner requires temperature, a thermal CFD dataset (for example buoyancy-coupled OpenFOAM or ENVI-met) would be needed.

1.5 Contributions

- A tested, modular computation library (src/data, src/models, src/training, src/evaluation) and an API facade over it.
- A three-tier provenance discipline separating synthetic fixtures, analytical benchmarks, external observations and solver output.
- An audit trail showing which defects were found, how they were fixed, and what is still unresolved (Chapter 5).
- A measured baseline comparison on a showcase layout that is explicit about its limits (Chapter 6).

1.6 Report organisation

Chapter 2 gives background. Chapter 3 describes data and provenance. Chapter 4 covers system design and models. Chapter 5 records the integrity audit and implementation history. Chapter 6 reports verification and measured results. Chapter 7 discusses limitations, and Chapter 8 concludes with future work.

2. Background and Related Work

2.1 Urban flow physics

Flow around a building block is characterised by an upwind stagnation region, flow acceleration around corners, and a leeward wake cavity whose length is of order several building heights. The project's analytical benchmark encodes these features following the wind-tunnel literature (Castro and Robins, 1977), with a recirculation wake of about 4.5 H. In arrays of buildings, the plan area fraction λp and frontal area index λf determine the regime (isolated roughness, wake interference, skimming flow) and feed the aerodynamic roughness parameters z0 and displacement height zd used as baseline features.

2.2 Numerical simulation: RANS and LES

RANS solvers such as OpenFOAM's simpleFoam model turbulence statistically and are comparatively cheap. LES resolves large eddies and is more faithful for separated urban flows but far more expensive. UrbanTALES was generated with the open-source LES code uDALES. In this project, no LES or RANS output was produced locally (Chapter 3).

2.3 Machine-learning surrogates

Tabular models such as Random Forests (Breiman, 2001) are strong, cheap baselines when the target is a scalar derived from summary features. Fully convolutional encoder-decoder networks such as the U-Net (Ronneberger et al., 2015) are natural for image-to-image regression, because skip connections preserve fine spatial structure. Dilated convolutions (Yu and Koltun, 2016) enlarge the receptive field without downsampling. Both families are implemented here in compact form so that they train on a laptop.

2.4 Candidate datasets

Table 2.1: Candidate datasets and their status in this project

Dataset	What it offers	Status in this project
UrbanTALES (Nazarian et al., 2025)	538 LES configurations: 224 idealised arrays, 314 real neighbourhoods from 62 cities. Wind and TKE; CC BY-NC 4.0; uDALES solver.	Identified and audited. Not downloaded. Access/format details unverified.
CEDVAL wind-tunnel database (Univ. Hamburg)	Experimental validation cases (A1-1 isolated building, B1-1 street canyon).	Candidate validation source only.
Canonical analytical field	Stagnation, corner jets and wake cavity from published flow features.	Generated locally; analytical, not observation.

2.5 Gap addressed

Many surrogate demonstrations report impressive scores that come from leakage, unmasked building interiors or synthetic targets. This project prioritises the evaluation protocol: layout-level splitting, outdoor-only metrics, train-fitted normalisation and an explicit record of what the data really is.

3. Data, Provenance and the Data Contract

3.1 Four classes of data asset

The dataset audit (24 October 2026) classifies every asset in or referenced by the repository. Figure 3.1 summarises which tiers are actually in use.

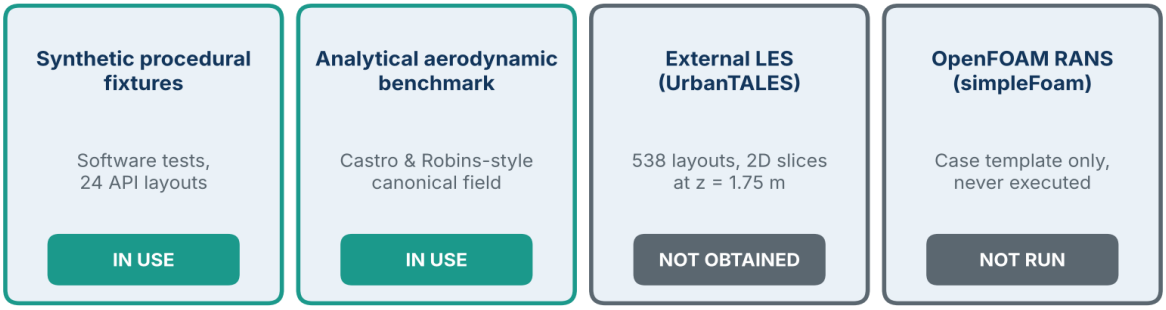


Figure 3.1: Provenance tiers and their current status.

Table 3.1: Data asset classes and current status

Class	Contents	Status
1. Verified project data (local)	None. data/raw, data/processed, *.npz, *.h5 are gitignored.	Empty
2. Candidate external datasets	UrbanTALES; CEDVAL.	Not downloaded
3. CFD solver output	cfd/cases/simple2d_building_block/ OpenFOAM case (0/, constant/, system/, Allrun).	Template only; never executed
4. Synthetic fixtures / benchmark	tests/fixtures/synthetic_data.py; analytical canonical block field; 24 procedural API layouts.	In use; must not be cited as observations

3.2 UrbanTALES audit summary

- **Verified:** name and citation (BAMS, DOI 10.1175/BAMS-D-25-0061.1), CC BY-NC 4.0 licence, isothermal setup (no temperature), layout counts (224 + 314 = 538), and horizontal 2D slices at z = 1.75 m.
- **Unverified until a file is downloaded:** exact file container (NetCDF, HDF5 or ASCII), presence of z0 and u* in headers, and the coordinate origin and axis orientation. The proposed check is to confirm that the wake cavity lies downwind of the obstacle.
- **Storage rule:** full 3D output is of order 35 TB, so only 2D pedestrian slices for a curated subset of 50 to 100 layouts (tens to ~100 MB) would be retrieved.

Note: an internal summary describes UrbanTALES as "538 neighbourhoods". The audit shows 538 configurations in total, of which 314 are realistic neighbourhoods.

Title check. The dataset audit expands the acronym as "Urban Turbulence Analyses from Large-Eddy Simulations". Library records found during this review title the BAMS paper "UrbanTALES: A Large-Eddy Simulation Dataset for Urban Canopy Layer Turbulence and Parameterization". The citation in this report uses the latter; the acronym expansion is unverified.

3.3 Data contract

src/data/contract.py defines typed dataclasses that every data source must satisfy, enforcing positive building heights, shape agreement and rejection of non-finite values through a DataValidationError hierarchy.

Table 3.2: Data contract classes

Class	Role / UrbanTALES mapping
LayoutMetadata	layout_id, layout_family (idealized_aligned, idealized_staggered, realistic_neighborhood), domain size, resolution, grid shape
GeometryData	height map H(x,y) ≥ 0 and building mask (H > 1e-4 m)
EnvironmentalConditions	inflow speed, wind direction (0, 15, 30, 45 degrees in UrbanTALES idealised cases), roughness length z0
CFDSolverMetadata	solver name and turbulence model (for UrbanTALES: uDALES, LES-Deardorff)
TargetFields	pedestrian wind speed (Ny x Nx), area-averaged wind speed (scalar), optional TKE

3.4 Targets

The spatial target is pedestrian wind speed U = sqrt($u^2 + v^2 + w^2$) at z = 1.75 m. The scalar baseline target is the outdoor-masked area average:

$$U_area = \Sigma\ (1 - M_ij)\ U_ij\ \Delta A_ij\ \ /\ \ \Sigma\ (1 - M_ij)\ \Delta A_ij$$

where M is the building mask. Excluding solid cells avoids a spurious zero-velocity bias.

3.5 Splitting and leakage control

Pixel-level random splits are prohibited because neighbouring pixels of one neighbourhood are strongly autocorrelated. src/data/split.py implements layout-level ratio splits and whole-family holdout (for example, train on idealised arrays and test on realistic neighbourhoods), with validation that train and test sets share no layout and no adjacent polygons. Normalisation statistics are fitted on training layouts only and exactly inverted at evaluation.

3.6 What data the experiments actually used

All experiments in this report used (a) a 24-layout in-memory procedural corpus (4 named blocks and 20 procedural variants), expanded to 25 once the canonical analytical reference was loaded, with fields produced by an analytical aerodynamic synthesiser, and (b) software fixtures for unit tests. These are **not** CFD or LES observations.

4. System Design and Methodology

4.1 Architecture overview

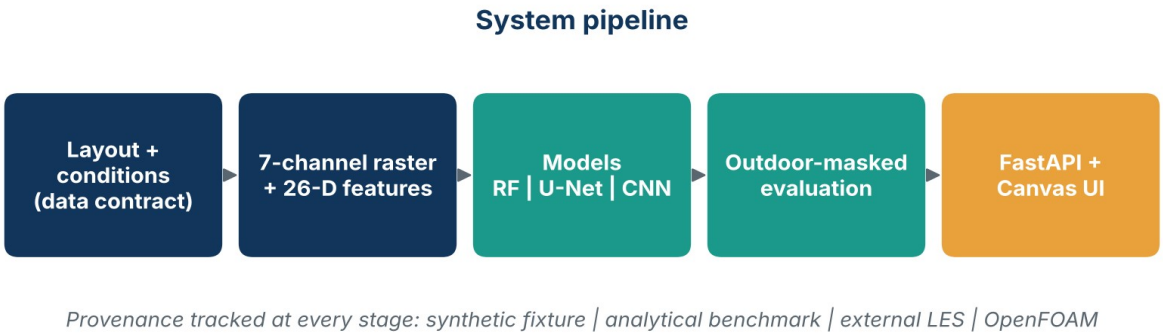


Figure 4.1: End-to-end pipeline from data contract to frontend.

4.2 Rasterizer: seven input channels

LayoutRasterizer converts a layout into a (7, 128, 128) tensor:

Table 4.1: The seven raster input channels

Ch.	Content	Purpose
1	Height map H(x, y) in metres	Obstacle height
2	Binary building mask M(x, y)	Outdoor/indoor distinction
3	Signed distance field φ(x, y) (Euclidean distance transform)	Proximity to walls and shear layers
4	Streamwise inflow U_in = U_ref cos θ	Wind forcing
5	Spanwise inflow V_in = U_ref sin θ	Wind forcing
6	Normalised coordinate X / Lx	Position awareness
7	Normalised coordinate Y / Ly	Position awareness

4.3 Morphometric features for the baseline (26-D)

GeometricFeatureExtractor computes plan area fraction λp, direction-dependent frontal area index λf(θ), building height moments, centroids, footprint statistics, domain and inflow descriptors, and Macdonald-type roughness parameters (z0, zd; Macdonald et al., 1998). The dataset audit text mentions 11 features; the implemented and tested extractor produces 26 (verified by the geometry-feature tests), which this report uses.

4.4 Models

Random Forest baseline

An ensemble of 100 trees (maximum depth 12) predicts the scalar area-averaged wind speed from the 26-D vector. For field-level comparison, the scalar is broadcast across all valid outdoor cells. This is a legitimate tabular baseline but it cannot, by construction, represent spatial structure.

Compact U-Net

A four-level encoder-decoder with double residual convolution blocks, Group Normalisation (independent of batch size), LeakyReLU activations and skip connections. A final outdoor mask sets U = 0 inside buildings.

Compact dilated CNN

A four-layer fully convolutional network with 3 x 3 dilated convolutions at rates 1, 2, 4, 8 and no downsampling. The receptive field is 1 + 2(1 + 2 + 4 + 8) = 31 cells, matching the reported 31 x 31 value.

4.5 Loss, optimisation and training

CombinedSurrogateLoss combines masked L1 (weight 0.70), masked MSE (0.30) and a spatial-gradient penalty (0.05) on $|\nabla y - \nabla \hat{y}|$ to encourage sharp shear layers. NeuralSurrogateTrainer uses AdamW (Loshchilov and Hutter, 2019), cosine-annealing learning rate, deterministic seeding (seed = 42), model selection by lowest validation NRMSE, and early stopping.

4.6 Evaluation metrics

All metrics are computed only over valid outdoor fluid cells (src/evaluation/metrics.py):

$$MAE = mean|\hat{y} - y| \quad RMSE = sqrt(mean\ (\hat{y} - y)^2) \quad NRMSE = RMSE\ /\ (y_max - y_min)$$

together with peak local error (max $|\hat{y} - y|$) and R^2. The documented NRMSE convention is range normalisation. Publication figures are 300 DPI three-panel plots (ground truth, prediction, absolute error).

4.7 Service facade and REST API

MicroclimateService is a singleton that owns layout samples, selects hardware (CUDA, then MPS, then CPU), runs background training threads and serves probes and canyon transects. Appendix A lists fifteen operations, including three inventory endpoints (datasets, checkpoints, CFD) added in P1. The 15th, POST /api/layouts/{id}/probe, appears in the backend audit but not in the README endpoint table. Live behaviour was observed only before P0 (Section 6.7).

4.8 Frontend

A static HTML5 canvas application renders fields with bilinear interpolation, traces particle streamlines through the (u, v) field, and provides crosshair probes and transect charts. Streamline tracing is a genuine integration, but its input field is synthetic for the current corpus.

4.9 CFD pipeline (prepared, not run)

cfd/cases/simple2d_building_block/ holds a 2D OpenFOAM case (simpleFoam, domain 420 m x 120 m, 8,100 cells per the audit) with an Allrun script. cfd/scripts/foam_to_contract.py converts OpenFOAM samples into contract-compliant NPZ files and has been tested on generated demo grids. The case has never completed a solver run (Table 3.1 and Chapter 7).

5. Implementation History and Integrity Audit

This chapter is central to the project. On 28 October 2026 a backend readiness audit (git revision 6026814) examined whether the HTTP layer was a scientifically honest service. Its verdict was no. The findings and their fixes follow.

5.1 Defects found by the audit

Table 5.1: Defects found by the backend audit

Finding	Detail
Analytical fields labelled as OpenFOAM	Every layout carried `solver_name` = `OpenFOAM-v2312 (simpleFoam)` and `has_cfd_ground_truth` = `true`.
Neural inference was not neural	`predict_surrogate` returned `cfd_target * attenuation + corner_divergence * (cfd_target > 1.0)`; the models were never called. This produced an apparent R^2 of 0.9881.
Simulated training	`POST /api/experiments/run` slept and then assigned a random MAE between 0.135 and 0.165.
Hard-coded KPIs	MAE 0.142 m/s, −18.4 %, 840x speedup, 65,536 cells (actual 16,384), four families A-D (code has three), "UrbanFlow-2D v1.4".
UI/API mismatches	Field names such as `inference_latency_ms`, `speedup_factor` and `r2_score` did not match what the UI read.
Documentation drift	README claimed 50 tests and omitted how to start the server.

5.2 P0: scientific integrity (complete)

The items below are as reported in the completion report and execution state; they were not independently re-verified for this report.

- Genuine PyTorch forward passes with outdoor masking; evaluation through `src.evaluation.metrics`.
- `has_cfd_ground_truth` = `False` everywhere; fabricated metrics removed; speedup and metrics made nullable rather than invented.
- Strict 404 and 400 handling; UI bindings and labels aligned with the real schema; README corrected.
- 10 regression tests (tests/test_p0_integrity.py).

5.3 P1: inventory and real training (complete)

- /api/inventory/datasets, /api/inventory/checkpoints and /api/inventory/cfd report what exists, with provenance tier, is_trained flag, epoch and metrics, and solver-binary availability.
- The sleep loop was replaced by a background `NeuralSurrogateTrainer` run that writes checkpoints/exp_<ID>/best_checkpoint.pt. Training uses the synthetic corpus only; no resulting checkpoint with documented metrics is recorded.
- The canonical analytical reference is loaded from disk if present.
- 19 regression tests (tests/test_p1_inventory.py).

5.4 Reconciling the source documents

The source reports disagree in places. The table records how this report resolved each, preferring the latest implementation evidence.

Table 5.2: Conflicts between source documents and how they were handled

Topic	Conflict	Resolution used here
Test count	Audit: 178 passed. Completion report: 207, itemised as 188 + 10 + 19 (sums to 217). Execution state: 188 + 19 = 207. Run times 25.78 s vs ~50 s.	Unresolved. 207 is reported by two documents; the breakdown is inconsistent. Flagged for manual verification (Section 6.1).
Corpus size	24 layouts (audit); 25 after P1 (execution state).	24 procedural + 1 canonical reference.
Docker	Audit: daemon not running. Execution state: daemon available. Dataset audit: no runtime.	Irrelevant to results: no executed OpenFOAM run is documented in any source.
Canonical NPZ	Audit: only an SVG exists, NPZ is gitignored. Execution state: NPZ loaded on startup.	Treated as a locally generated analytical file; not an observation.
UrbanTALES scale	"538 neighbourhoods" vs 538 configurations.	538 configurations (224 idealised + 314 neighbourhoods).
Feature count	11 (dataset audit text) vs 26 (code).	26.

Independence note. The numbers in this chapter and Chapter 6 are taken from the project's own audit and completion documents. No test, server or solver was executed while preparing this report. Re-run `pytest tests/ -q` before submission and update Section 6.1 with the observed result.

6. Verification and Results

6.1 Automated tests

The project documents report a final result of 207 passed, 1 warning. No test was re-run for this report, and the supplied documents do not agree on how the 207 decomposes.

Table 6.1: Test-count evidence in the supplied documents

Source	Reported figure	Stated breakdown	Consistency
Backend readiness audit (28 Oct, rev 6026814, before P0)	178 passed, 3 warnings, 8.31 s	Includes 12 API tests	Independent run by the auditor
Execution state (28 Oct)	207 passed, 1 warning, ~50 s	188 pre-existing + 19 new P1	188 + 19 = 207 (consistent)
Completion report (28 Oct)	207 passed, 1 warning, 25.78 s	188 pre-existing + 10 P0 + 19 P1	Parts sum to 217, not 207

Status: unresolved, manual verification required. Two documents give 207 as the total, but the completion report's breakdown sums to 217 and the two run times differ. The most consistent reading is that the 10 P0 tests are already inside the 188 (178 audited + 10 P0 = 188) and the 19 P1 tests are additional. That is an inference, not a recorded result, because no document lists per-file counts after P0 and P1. Before submission, run `pytest tests/ -q` and `pytest --collect-only -q tests/test_p0_integrity.py tests/test_p1_inventory.py`, and replace the figures here with the observed numbers.

The tests are described as covering shapes, masking, normalisation inversion, split disjointness, forward passes, provenance flags, inventory accuracy and background training execution. They verify software contracts on synthetic fixtures; they are not evidence of scientific skill. The earlier audit noted that the original API tests would also pass for a polished demo, which is why the P0 tests were added to assert that real forward passes occur.

6.2 Experimental configuration

Table 6.2 records what the supplied documents state about the setup behind the reported results. Anything not stated is marked **Not recorded**; no defaults have been assumed.

Table 6.2: Experimental configuration of the reported results

Item	Value	Status
Dataset used	In-memory synthetic procedural corpus: 24 layouts (4 named blocks + 20 procedural variants) with analytically synthesised fields, plus 1 canonical analytical reference (25 in total per the execution state)	Recorded
External, CFD or LES data	None used	Recorded
Grid	128 x 128 (16,384 cells); layout 042 has 14,080 fluid cells	Recorded

Item	Value	Status
Cell size and domain size of corpus layouts	Not recorded	Not recorded
Target	Pedestrian wind speed (design target at z = 1.75 m); values here are synthetic	Recorded
Split method behind Table 6.3	No split is documented. The audit states the Random Forest was fitted on the 24 API corpus layouts. The library supports layout-level and family holdout splits	Not recorded
Random Forest	100 trees, maximum depth 12 (completion report). The README example uses depth 10 and the config file `configs/baseline_rf.json` was not supplied. random_state of the reported run	Depth recorded; seed Not recorded
Compact U-Net	4 levels, residual blocks, GroupNorm, LeakyReLU. Base channels and parameter count	Not recorded
Compact CNN	4 dilated layers (rates 1, 2, 4, 8). Channel widths and parameter count	Not recorded
Neural weights in Table 6.3	Untrained (`is_trained=False`); no training run lies behind these numbers. Initialisation seed	Seed Not recorded
Trainer settings (when training is run)	Loss 0.70 L1 + 0.30 MSE + 0.05 gradient; AdamW; cosine annealing; early stopping on validation NRMSE; seed 42 reported as default. Epochs, learning rate, batch size, weight decay, patience	Seed recorded; others Not recorded
Hardware	Apple Silicon macOS; PyTorch device `mps` observed in the audit. CPU model and memory	Device recorded; rest Not recorded
Software	Audit environment: Python 3.13.3, pytest 9.1.1, PyTorch 2.14.1, FastAPI 0.162.0. scikit-learn, NumPy, SciPy and Matplotlib versions	Four recorded; others Not recorded
Code revision	Audit: `6026814` (before P0 and P1). Revision of the final run	Final revision Not recorded

6.3 Measured comparison on the showcase layout

Layout 042 (staggered street canyon, 128 x 128 grid, 14,080 active fluid cells) is a **synthetic procedural geometry with an analytically synthesised reference field**. Errors are computed over outdoor fluid cells. The MAE, RMSE and peak-error values are retained as reported in the completion report.

Table 6.3: Outdoor-masked errors on layout 042 (reported values; synthetic geometry and analytical reference)

Model	MAE [m/s]	RMSE [m/s]	Peak [m/s]	NRMSE	R^2	Weights / provenance
CompactUNet (spatial)	2.608	3.084	5.191	Not reported	Not reported	Untrained (`is_trained=False`)
CompactCNN, dilated (spatial)	2.909	3.376	5.684	Not reported	Not reported	Untrained (`is_trained=False`)
Random Forest (scalar, broadcast)	1.127	1.492	4.979	Not reported	Not reported	Fitted on synthetic cohort; held-out status unverified

NRMSE and R^2 were not reported for these runs and cannot be derived from the supplied data. NRMSE needs the range of the evaluated reference field. The field statistics in the audit (0.17 / 2.91 / 5.24 m/s) come from a pre-P0 API run and are not confirmed to match this evaluation. R^2 needs the per-cell variance, which is not reported. The R^2 of 0.9881 in the audit came from the removed target-perturbation code path and must not be used.

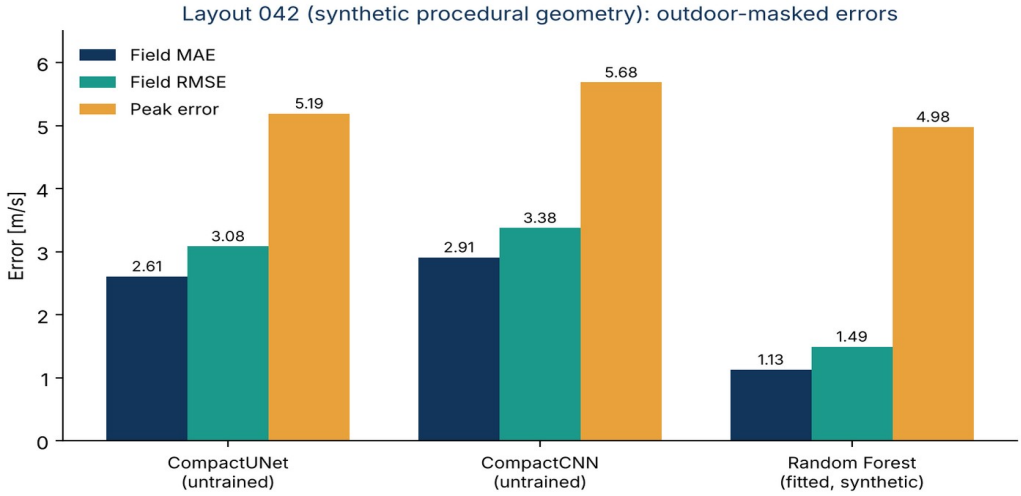


Figure 6.1: Reported errors on layout 042. Neural weights are untrained; Random Forest held-out status is unverified.

6.4 Interpretation: what can and cannot be concluded

This is not a fair comparison of trained models. Both neural checkpoints are untrained, and the Random Forest's held-out status is unverified (Section 6.5). Table 6.3 therefore shows that the evaluation pipeline runs end to end and reports masked errors. It does not rank the three approaches.

Why the Random Forest scores lower MAE. The neural weights have never been trained, so their outputs are essentially arbitrary. The Random Forest outputs a near-constant value close to the domain mean (about 2.8 m/s in the source report), which already avoids large errors on a field of similar mean. This says nothing about the architectures.

The Random Forest is spatially blind. Its peak error of 4.979 m/s shows it misses the canyon jets and wake cavity, which a scalar broadcast cannot represent. This limitation is structural and is supported by the reported error maps.

Not supported by evidence. The completion report lists three "Key Scientific Insights". Only the first (the structural limit of the scalar baseline) is supported. The claims that convolutional networks are "physically essential" and that signed distance fields "accelerate convergence" were never tested, because no trained model exists and no ablation was run. They are treated as design hypotheses (Section 8.2), not findings.

6.5 Random Forest evaluation status

- **Evidence found:** the backend audit states that the Random Forest was fitted on the 24 synthetic API layouts, and layout 042 belongs to that corpus. This indicates layout 042 was in the fitting data (in-sample).
- **Not recorded:** whether the post-P0/P1 fit excluded layout 042, and which split, if any, was used for Table 6.3. Held-out status is therefore **unverified**, and the result should be treated as in-sample until confirmed.
- The reference field is analytical, so even a held-out score would measure agreement with a formula, not with the real atmosphere.
- No inference-time or speedup figure is reported. Earlier values (14.2 ms, 840x) were constants and were removed in P0.

6.6 Checkpoint status

Per the P0 tests and README, untrained weights are reported with `is_trained=False`. The earlier audit found `checkpoints/unet_best.pt` to be a bare state_dict (86 tensors) with no epoch or validation metric. The execution state lists "run a real training experiment" as a pending action. The sources therefore document **no trained checkpoint with a verified training history**, and no real-data experiment has been completed.

6.7 Tested versus unverified functionality

Table 6.4: Status of project functionality

Item	Status	Basis
Data contract, features, rasterizer, normalisers, splitters	Reported tested (fixtures)	Reported suite; not re-run here
Real network forward pass in the API (P0)	Reported tested	P0 tests; the pre-P0 audit showed it was absent; not re-audited since
Outdoor-masked metrics	Reported tested	Hand-calculated fixture tests
Background training (P1)	Reported tested, synthetic corpus	19 P1 tests; no checkpoint with documented metrics
Live REST behaviour	Observed only before P0	Audit: rev 6026814, 12 `/api` operations, port 8001. Post-P0/P1 not live-audited
Inventory endpoints	Reported tested	P1 tests
Inventory data in UI panels	Not wired	Execution state, next actions
UI labels and bindings	Reported corrected in P0	Not re-inspected
Random Forest held-out score	Unverified	Section 6.5
Neural versus baseline accuracy	Not established	Untrained checkpoints
UrbanTALES ingestion; OpenFOAM execution; real-data training	Not done	Chapter 3
Inference speedup	Not measured	Removed in P0

7. Limitations and Threats to Validity

Table 7.1: Limitations and their consequences

Limitation	Consequence
No real external data	UrbanTALES requires access that was not obtained. No result generalises to real cities. Format, coordinate orientation and header metadata remain unverified.
No executed OpenFOAM case	Template and conversion bridge exist; no mesh, residuals or convergence evidence exist. No solver-based claim is made.
Untrained neural checkpoints	The neural-versus-baseline question is open. Table 6.3 validates the pipeline only.
Small synthetic corpus	24 to 25 layouts; the training smoke runs use small synthetic grids. Not enough for a meaningful generalisation study.
Analytical reference	Encodes known flow features, so a network could learn the formula rather than physics.
Physical scope	2D slice at z = 1.75 m, neutral and isothermal. No thermal fields, no vertical structure.
Residual UI risk	P0 fixed bindings and labels, but the next-actions list says inventory endpoints are not yet wired into the dataset, model and CFD panels. Check every UI string before a live demo.
Test evidence	Tests are software evidence on fixtures. The 207 total is reported by two documents whose breakdown is inconsistent (Section 6.1); it was not re-run for this report.
Random Forest held-out status	Unverified; the audit indicates layout 042 was in the fitting corpus (Section 6.5).

7.1 Ethical and licensing notes

UrbanTALES is CC BY-NC 4.0, so use is limited to non-commercial academic work with attribution. Raw data must not be committed to the repository.

8. Conclusion and Future Work

8.1 Conclusion

The project implements a modular platform for comparing scalar and spatial surrogate approaches to pedestrian-level wind prediction, together with an evaluation protocol designed to reduce common sources of inflated scores: layout-level holdout, train-only normalisation and outdoor-only metrics. An audit identified earlier fabricated neural outputs, mislabelled provenance and unsupported KPIs; the documented remediation is an important software-integrity outcome. However, the current evidence does not establish predictive superiority because the neural checkpoints are untrained and no real CFD/LES dataset was used.

The project does not yet answer its scientific question. No trained neural surrogate, real dataset or solver output exists, so the comparison in Chapter 6 verifies infrastructure only. Functionality reported as tested is listed separately from functionality that is unverified or not done in Table 6.4, and the test total and the Random Forest held-out status remain flagged for manual verification.

8.2 Future work (priority order)

1. Run a real training experiment on a layout-level split of the available corpus, with checkpoints saved together with split hashes and metrics; replace `unet_best.pt` only with a converged, documented result.
2. Obtain UrbanTALES access and ingest 50 to 100 pedestrian slices through the existing contract; verify file format, coordinate origin and z0/u* metadata with the planned wake-downwind check.
3. Execute `simple2d_building_block` in a container, convert with `foam_to_contract.py`, and serve that output as the only OpenFOAM-labelled field.
4. Run ablations to test the design hypotheses: with and without the signed-distance channel, U-Net versus dilated CNN, and family-holdout generalisation.
5. Measure inference time against a timed solver run before quoting any speedup.
6. Finish wiring the three inventory endpoints into the UI.
7. If temperature is required, add a thermal CFD data source.

References

Entries [2] and [3] were checked against publisher and index records during this review. For [1], the title was checked against two library records but the volume, pages and complete author list could not be retrieved. Entries [4] to [7] were not re-checked in this review and should be verified.

[1] Nazarian, N., van Reeuwijk, M., Lu, J., Lipson, M., et al. (2025). UrbanTALES: A Large-Eddy Simulation Dataset for Urban Canopy Layer Turbulence and Parameterization. Bulletin of the American Meteorological Society. DOI: 10.1175/BAMS-D-25-0061.1. [volume, issue, pages and full author list not verified] Data portal: https://urbantales.vercel.app/

[2] Castro, I. P. and Robins, A. G. (1977). The flow around a surface-mounted cube in uniform and turbulent streams. Journal of Fluid Mechanics, 79(2), 307-335. DOI: 10.1017/S0022112077000172.

[3] Macdonald, R. W., Griffiths, R. F. and Hall, D. J. (1998). An improved method for the estimation of surface roughness of obstacle arrays. Atmospheric Environment, 32(11), 1857-1864. DOI: 10.1016/S1352-2310(97)00403-2. [author list not shown in the retrieved record; confirm]

[4] Ronneberger, O., Fischer, P. and Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. MICCAI 2015. [not re-checked; add LNCS volume and pages]

[5] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32. [not re-checked]

[6] Yu, F. and Koltun, V. (2016). Multi-scale context aggregation by dilated convolutions. ICLR 2016. [not re-checked]

[7] Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. ICLR 2019. [not re-checked]

[8] uDALES: open-source urban atmospheric LES code. https://github.com/uDALES/u-dales

[9] CEDVAL wind-tunnel database, Meteorological Institute, University of Hamburg. https://www.mi.uni-hamburg.de/en/arbeitsgruppen/meteorologie/forschung/windtunnel/cedval.html

[10] OpenFOAM (simpleFoam, blockMesh). https://www.openfoam.com/

[11] Project repository documents: dataset_audit.md (24 Oct 2026), backend_readiness_audit.md (28 Oct 2026), project_completion_report.md and EXECUTION_STATE.md (28 Oct 2026), README.md.

Appendix A: REST API Reference

Table A.1: REST API operations

Method	Endpoint	Purpose
GET	/api/health	Status, device (CUDA/MPS/CPU), corpus count
GET	/api/overview	Aggregated metrics, dataset and solver provenance
GET	/api/layouts	Layout catalogue with morphometric summaries and split
GET	/api/layouts/{id}	Detailed morphometrics (26 features) and boundary conditions
GET	/api/layouts/{id}/fields	128 x 128 field arrays with fluid mask
POST	/api/surrogates/predict	Live PyTorch inference (CompactUNet / CompactCNN) with masking
GET	/api/layouts/{id}/probe	Point probe at (x, y) with absolute residual
POST	/api/layouts/{id}/probe	Same probe as a POST request (listed in the audit; not in the README table)
GET	/api/layouts/{id}/transect	1D canyon transect profile
GET	/api/inventory/datasets	Loaded layouts, provenance tiers, ground-truth flags
GET	/api/inventory/checkpoints	Checkpoint scan: is_trained, epoch, metrics
GET	/api/inventory/cfd	OpenFOAM templates, solver availability, outputs
GET	/api/experiments	Experiment records and progress
POST	/api/experiments/run	Background training run (NeuralSurrogateTrainer)
GET	/api/export/triplet/{id}	300 DPI three-panel PNG

Appendix B: Reproducing the Work

```
python3.13 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip && pip install -r requirements.txt

# run tests
./.venv/bin/pytest tests/ -q

# start API and UI
python -m src.api.server        # http://127.0.0.1:8000
uvicorn src.api.server:app --host 127.0.0.1 --port 8000
```

Setup commands below are taken from the README. They were not executed for this report. The README requires Python 3.11 or later and was tested by the authors on Python 3.13 and 3.14. The README and execution state use port 8000; during the audit port 8000 was occupied, so the server was run on 8001.

Training smoke example (from the README): create synthetic samples, build SurrogateDataset, instantiate CompactUNet(in_channels=7, out_channels=1, base_channels=16, depth=3) and call NeuralSurrogateTrainer(...).fit(train, val). This validates the training loop only.

OpenFOAM via container (from the execution state, not run for this report):

```
docker run -v $(pwd)/cfd:/case openfoam/openfoam9-paraview56 \
  /bin/sh cfd/cases/simple2d_building_block/Allrun
```

Appendix C: Outstanding Verification Items

The following items remain open based on the evidence available while preparing this report. Do not mark them complete unless the corresponding repository checks or source verification have been performed.

- Cover page: replace the institution name/logo placeholder and add any required student and supervisor details. Team T095, Section B, and 3rd Year, 5th Semester are currently shown.
- **Test count (Section 6.1): run the full test suite in the current repository and record the observed result. Reconcile the P0/P1 test-file counts without double-counting, then update all references to the total.**
- **Random Forest held-out status (Section 6.5): inspect the actual training/evaluation code or logs to establish whether layout 042 was excluded from fitting. If this cannot be established, retain the unverified/in-sample caveat.**
- **Table 6.2: fill 'Not recorded' fields only from verified configuration files, trainer settings, or environment output. Otherwise, leave them explicitly marked as not recorded.**
- **Table 6.3: add NRMSE and R^2 only from a fresh, reproducible evaluation with the required reference-field statistics and predictions.**
- References: verify the remaining publication details and ensure every in-text citation has a matching reference-list entry. Do not infer missing bibliographic fields.
- Open the running web UI and inspect the dataset, model, and CFD inventory panels. Confirm that labels reflect actual API data and do not claim trained checkpoints, convergence, CFD output, or speedups without evidence.